Singleton Design Pattern

Exercise 1 — Complete Answer with Java Source Code

Project: SingletonPatternExample

Topic: Creational Design Patterns | Java

Pattern Type	Creational
Goal	One instance, global access point
Key Principle	Private constructor + static getInstance()
Thread Safety	Handled via synchronized / volatile

STEP 1

Project Setup — SingletonPatternExample

Create a new Java project named **SingletonPatternExample**. The project will contain two Java files:

- **Logger.java** — the Singleton class
- **SingletonTest.java** — the test / driver class

```
SingletonPatternExample/
    src/
        Logger.java
        SingletonTest.java
```

STEP 2

Define the Singleton Logger Class

Key Rules of the Singleton Pattern

- **Private static instance** — holds the single object reference
- **Private constructor** — prevents external instantiation with `new`
- **Public static getInstance()** — the only way to get the object

Logger.java — Basic Singleton (Lazy Initialization)

```
// File: Logger.java

public class Logger {

// Step 1: Private static variable to hold the single instance
// 'null' at start — created only when first needed (lazy)
private static Logger instance = null;

// Step 2: Private constructor — prevents "new Logger()" from outside
private Logger() {
System.out.println("Logger instance created.");
}

// Step 3: Public static method — global access point
public static Logger getInstance() {
if (instance == null) { // create only if not yet created
instance = new Logger();
}
return instance;
}

// ■■ Logging methods ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public void log(String message) {
System.out.println("[LOG] " + message);
}

public void warn(String message) {
System.out.println("[WARN] " + message);
}
```

```
public void error(String message) {
System.out.println("[ERROR] " + message);
}
}
```

■ *Lazy initialization delays object creation until it is first needed, saving memory if the instance is never required.*

STEP 3

Thread-Safe Singleton Implementation

The basic version above is NOT safe when multiple threads call **getInstance()** simultaneously — two threads could each see **instance == null** and create two objects, violating the pattern. Use one of the approaches below in production code.

Option A — Double-Checked Locking (Recommended)

```
// File: Logger.java (Thread-Safe – Double-Checked Locking)

public class Logger {

// 'volatile' ensures visibility across threads
private static volatile Logger instance = null;

private Logger() {
System.out.println("Logger instance created.");
}

public static Logger getInstance() {
if (instance == null) { // 1st check (no lock)
synchronized (Logger.class) { // lock the class
if (instance == null) { // 2nd check (with lock)
instance = new Logger();
}
}
}
return instance;
}

public void log(String message) {
System.out.println("[LOG] " + message);
}

public void warn(String message) {
System.out.println("[WARN] " + message);
}

public void error(String message) {
System.out.println("[ERROR] " + message);
}
}
```

Option B — Bill Pugh (Static Inner Helper Class) — Simplest & Best

```
// File: Logger.java (Thread-Safe – Bill Pugh Singleton)

public class Logger {

private Logger() {}

// Inner static class – loaded only when getInstance() is first called
private static class LoggerHolder {
private static final Logger INSTANCE = new Logger();
}
```

```
public static Logger getInstance() {
return LoggerHolder.INSTANCE; // JVM guarantees thread safety
}

public void log(String message) {
System.out.println("[LOG] " + message);
}

public void warn(String message) {
System.out.println("[WARN] " + message);
}

public void error(String message) {
System.out.println("[ERROR] " + message);
}
}
```

■ *Bill Pugh Singleton is preferred — no synchronization overhead, lazy loading, and thread safety guaranteed by the JVM class loader.*

STEP 4

Test the Singleton Implementation

The test class verifies that **no matter how many times getInstance()** is called, the **same object** is always returned. We confirm this by comparing hash codes and using the **==** operator.

SingletonTest.java

```
// File: SingletonTest.java

public class SingletonTest {

public static void main(String[] args) {

System.out.println("=== Singleton Pattern Test ===\n");

// ■■ Test 1: Obtain two references ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Logger logger1 = Logger.getInstance();
Logger logger2 = Logger.getInstance();

// ■■ Test 2: Compare with == (reference equality) ■■■■■■■■■■■■■■■■■■■
System.out.println("Are both references the same object?");
if (logger1 == logger2) {
System.out.println(" PASS: logger1 == logger2 (same instance)\n");
} else {
System.out.println(" FAIL: Different instances created!\n");
}

// ■■ Test 3: Compare hash codes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
System.out.println("Hash code of logger1 : " + logger1.hashCode());
System.out.println("Hash code of logger2 : " + logger2.hashCode());
System.out.println("Hash codes match : " + (logger1.hashCode() == logger2.hashCode()));

System.out.println();

// ■■ Test 4: Use logging methods ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
System.out.println("--- Logging via logger1 ---");
logger1.log("Application started.");
logger1.warn("Low memory warning.");
logger1.error("Null pointer encountered.");

System.out.println();
System.out.println("--- Logging via logger2 ---");
logger2.log("User login successful.");
logger2.warn("Session about to expire.");

System.out.println();

// ■■ Test 5: Third reference from a helper method ■■■■■■■■■■■■■■■■■■■■
Logger logger3 = getLoggerFromHelper();
System.out.println("logger1 == logger3 : " + (logger1 == logger3));
System.out.println("\n=== Test Complete ===");
}

// Simulates getting logger from a different part of the application
```

```
private static Logger getLoggerFromHelper() {
return Logger.getInstance();
}
}
```

Expected Console Output

```
=== Singleton Pattern Test ===

Logger instance created.
Are both references the same object?
PASS: logger1 == logger2 (same instance)

Hash code of logger1 : 366712642
Hash code of logger2 : 366712642
Hash codes match : true

--- Logging via logger1 ---
[LOG] Application started.
[WARN] Low memory warning.
[ERROR] Null pointer encountered.

--- Logging via logger2 ---
[LOG] User login successful.
[WARN] Session about to expire.

logger1 == logger3 : true

=== Test Complete ===
```

■ *"Logger instance created." prints only ONCE — proving a single object is shared across all calls to getInstance().*

KEY CONCEPTS

How Each Part Enforces the Singleton Pattern

Code Element	Why It's Needed
`private static Logger instance`	Holds the one shared object; static means it belongs to the class, not an object
`private Logger()`	Blocks any class from calling 'new Logger()' — only Logger itself can create one
`public static getInstance()`	Single access point; returns existing instance or creates it on first call
`if (instance == null)`	Lazy check — avoids creating an object until truly needed
`volatile keyword`	Prevents CPU/compiler reordering in multi-threaded environments
`synchronized block`	Ensures only one thread creates the instance even under concurrency
`logger1 == logger2`	== compares memory addresses; true means both variables point to same object
`hashCode()`	JVM default hashCode is based on memory address — identical codes confirm same object

Singleton vs Regular Class — Side by Side

Feature	Regular Class	Singleton Class
Instantiation	new MyClass()	MyClass.getInstance()
Number of objects	Unlimited	Exactly one
Constructor	public	private
Instance storage	Local variable	Static class variable
Thread safety	Not applicable	Must be ensured explicitly
Use case	Entities (Student, Order)	Logger, Config, Cache, DB Pool

UML Class Diagram (Singleton)

```
+-----------------------------------------+
| Logger |
+-----------------------------------------+
| - instance : Logger {static} |
+-----------------------------------------+
| - Logger() | <-- private constructor
| + getInstance() : Logger {static} | <-- public access point
| + log(message : String) : void |
| + warn(message : String) : void |
| + error(message : String) : void |
+-----------------------------------------+

Client code:
Logger.getInstance() ■■> returns the single instance
```

Summary

The Singleton Pattern ensures a class has **only one instance** and provides a **global access point** to it.

Three things make it work: a **private constructor**, a **private static instance variable**, and a **public static getInstance() method**.

For multi-threaded Java applications, use the **Bill Pugh** static inner class approach or **double-checked locking with volatile** for the best balance of safety and performance.